Task 2: Content-Based Image Retrieval (CBIR)

```
import pandas as pd
import numpy as np
import os

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from PIL import Image
import random
```

Dataset & Preprocessing

```
data_df = pd.read_csv("./nga_dataset/nga_paintings_metadata.csv")
print(data_df.columns)
```

```
Index(['objectid', 'uuid_x', 'accessioned', 'accessionnum',
'locationid',
       'title', 'displaydate', 'beginyear', 'endyear',
'visualbrowsertimespan',
       'medium', 'dimensions', 'inscription', 'markings',
       'attributioninverted', 'attribution', 'provenancetext',
'creditline',
       'classification', 'subclassification',
'visualbrowserclassification',
       'parentid', 'isvirtual', 'departmentabbr', 'portfolio',
'series',
       'volume', 'watermarks', 'lastdetectedmodification',
'wikidataid',
       'customprinturl', 'uuid_y', 'iiifurl', 'iiifthumburl',
'viewtype',
       'sequence', 'width', 'height', 'maxpixels', 'openaccess',
'created',
       'modified', 'depictstmsobjectid', 'assistivetext'],
      dtype='str')
```

```
portrait_keywords = [
    'portrait', 'head', 'face', 'woman', 'man', 'boy', 'girl',
'madonna',
    'saint', 'lady', 'gentleman', 'child', 'king', 'queen', 'duke',
    'self-portrait', 'family', 'figure', 'nude', 'bather', 'peasant',
    'soldier', 'christ', 'virgin', 'angel', 'apostle', 'pope', 'doge'
]

landscape_keywords = [
    'landscape', 'river', 'mountain', 'valley', 'view', 'scene',
```

```
'sea',
    'beach', 'coast', 'forest', 'wood', 'tree', 'field', 'meadow',
    'waterfall', 'lake', 'pond', 'harbor', 'bay', 'ruins',
'cityscape',
    'bridge', 'canal', 'farm', 'sunset', 'sunrise'
]

still_life_keywords = [
    'still life', 'flowers', 'fruit', 'bowl', 'vase', 'basket',
'glass',
    'table', 'plate', 'dish', 'bottle', 'cup', 'bread', 'cheese',
'wine',
    'meat', 'fish', 'books', 'skull', 'bouquet', 'peaches', 'apples',
'grapes'
]

def pseudo_labeling(title):
    if not isinstance(title,str): return -1

    title = title.lower()

    if any(word in title for word in portrait_keywords): return 0
    elif any(word in title for word in landscape_keywords): return 1
    elif any(word in title for word in still_life_keywords): return 2
    else: return -1

data_df['pseudo_label'] = data_df['title'].apply(pseudo_labeling)

labeled_df = data_df[data_df['pseudo_label'] != -1].copy()
print(f"Paintings successfully pseudo-labeled: {len(labeled_df)}")

img_dir = "nga_dataset/images"
labeled_df['img_path'] = labeled_df['objectid'].apply(lambda x:
os.path.join(img_dir,f"{x}.jpg"))

final_df =
labeled_df[labeled_df['img_path'].apply(os.path.exists)].copy()
print(f"Final usable dataset size: {len(final_df)}")

print("\nClass Distribution (0: Portrait, 1: Landscape, 2: Still
Life):")
print(final_df['pseudo_label'].value_counts())
```

```
Paintings successfully pseudo-labeled: 1670
Final usable dataset size: 1670

Class Distribution (0: Portrait, 1: Landscape, 2: Still Life):
pseudo_label
0    1065
1     400
```

```
2     205
Name: count, dtype: int64
```

```
class TripletDataset(torch.utils.data.Dataset):
    def __init__(self,df,transform=None):
        self.df = df
        self.transform = transform

        self.portraits = df[df['pseudo_label']==0]
['img_path'].tolist()
        self.landscapes = df[df['pseudo_label']==1]
['img_path'].tolist()
        self.still_life = df[df['pseudo_label']==2]
['img_path'].tolist()

    def __len__(self):
        return len(self.df)

    def __getitem__(self,idx):
        row = self.df.iloc[idx]
        anchor_path = row['img_path']
        anchor_label = row['pseudo_label']

        positive_list,negative_list=[],[]

        if(anchor_label==0):
            positive_list = self.portraits
            negative_list = self.landscapes + self.still_life
        elif(anchor_label==1):
            positive_list = self.landscapes
            negative_list = self.portraits + self.still_life
        else:
            positive_list = self.still_life
            negative_list = self.portraits + self.landscapes

        pos_path = random.choice(positive_list)
        neg_path = random.choice(negative_list)
        while(pos_path==anchor_path): pos_path =
random.choice(positive_list)

        anchor_img = Image.open(anchor_path).convert('RGB')
        pos_img = Image.open(pos_path).convert('RGB')
        neg_img = Image.open(neg_path).convert('RGB')

        anchor_img = self.transform(anchor_img)
        pos_img = self.transform(pos_img)
        neg_img = self.transform(neg_img)

        return anchor_img,pos_img,neg_img
```

```
class GalleryDataset(torch.utils.data.Dataset):
    def __init__(self,df,transform=None):
        self.df = df
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self,idx):
        row = self.df.iloc[idx]
        path = row['img_path']
        label = row['pseudo_label']

        img = Image.open(path).convert('RGB')
        img = self.transform(img)

        return (img,label)


eval_transform = torchvision.transforms.Compose(
    [
        torchvision.transforms.Resize(256),
        torchvision.transforms.CenterCrop(224),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize(mean=[0.485, 0.456, 0.406],
std=[0.229, 0.224, 0.225])
    ]
)

train_transform = torchvision.transforms.Compose(
    [
        torchvision.transforms.Resize(256),
        torchvision.transforms.RandomCrop(224),
        torchvision.transforms.RandomHorizontalFlip(),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize(mean=[0.485, 0.456, 0.406],
std=[0.229, 0.224, 0.225])
    ]
)

dataset = TripletDataset(df=final_df,transform=train_transform)
anchor,pos,neg = dataset[0]
print(f"Anchor shape: {anchor.shape}")
print(f"Positive shape: {pos.shape}")
print(f"Negative shape: {neg.shape}")
```

```
Anchor shape: torch.Size([3, 224, 224])
Positive shape: torch.Size([3, 224, 224])
Negative shape: torch.Size([3, 224, 224])
```

Model 1

Why a **Custom ResNet** as a Siamese Feature Extractor ?

- **Hierarchical Spatial Features**: Convolutional Neural Networks (CNNs) excel at building understanding pixel-by-pixel. By reusing the **custom ResNet architecture** from Task 1, the model inherently captures the local textural nuances such as specific brushstroke techniques and edge structures that are critical for differentiating artistic styles.

- **The Siamese Projection**: Standard classification networks predict discrete, mutually exclusive classes. However, Content-Based Image Retrieval (CBIR) requires projecting artworks into a continuous embedding space. We adapt the ResNet into a Siamese Network by stripping its final classification head. Instead, we use global average pooling to crush the spatial feature maps, acting purely as a spatial projector to map the image onto a 256-dimensional L_2 normalized hypersphere.

- **Preventing Representation Collapse**: If we used a standard contrastive loss (comparing only two images at a time), the network might suffer from representation collapse lazily learning to output the exact same vector for every single portrait to minimize loss. Triplet Margin Loss prevents this geometry collapse by evaluating three images simultaneously.

- **The Triplet Margin Objective**: This loss function forces the network to dynamically organize the internal geometry of the embedding space. It guarantees that the distance between the Anchor and the Positive ($d(a, p)$) is strictly smaller than the distance between the Anchor and the Negative ($d(a, n)$) by a rigidly defined mathematical margin (α).

$$L = max(d(a, p) - d(a, n) + \alpha, 0)$$

```
class ResidualBlock(nn.Module):
    def __init__(self, inchannel, outchannel, stride=1):
        super(ResidualBlock, self).__init__()
        self.left = nn.Sequential(
            nn.Conv2d(inchannel, outchannel, kernel_size=3,
stride=stride, padding=1, bias=False),
            nn.BatchNorm2d(outchannel),
            nn.ReLU(inplace=True),
            nn.Conv2d(outchannel, outchannel, kernel_size=3, stride=1,
padding=1, bias=False),
            nn.BatchNorm2d(outchannel)
        )
        self.shortcut = nn.Sequential()
        if stride != 1 or inchannel != outchannel:
```

```
            self.shortcut = nn.Sequential(
                nn.Conv2d(inchannel, outchannel, kernel_size=1,
stride=stride, bias=False),
                nn.BatchNorm2d(outchannel)
            )

    def forward(self, x):
        out = self.left(x)
        out = out + self.shortcut(x)
        out = F.relu(out)

        return out


class ResNet(nn.Module):
    def __init__(self, ResidualBlock):
        super(ResNet, self).__init__()
        self.inchannel = 64
        self.conv1 = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU()
        )
        self.layer1 = self.make_layer(ResidualBlock, 64, 2, stride=1)
        self.layer2 = self.make_layer(ResidualBlock, 128, 2, stride=2)
        self.layer3 = self.make_layer(ResidualBlock, 256, 2, stride=2)

        self.layer4 = self.make_layer(ResidualBlock, 512, 2, stride=2)


    def make_layer(self, block, channels, num_blocks, stride):
        strides = [stride] + [1] * (num_blocks - 1)
        layers = []
        for stride in strides:
            layers.append(block(self.inchannel, channels, stride))
            self.inchannel = channels
        return nn.Sequential(*layers)

    def forward(self, x):
        out = self.conv1(x)
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        return out


class EmbeddingNet(torch.nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.fc = nn.Linear(512, 256)
        self.AAP = nn.AdaptiveAvgPool2d((1,1))
        self.ResNet = ResNet(ResidualBlock)

    def forward(self,input):
        features = self.ResNet(input)
        features = self.AAP(features)
        features = features.flatten(start_dim=1)
        features = self.fc(features)
        features = F.normalize(features,p=2,dim=1)
        return features
```

Model init

```
model = EmbeddingNet().to('cuda')
dataloader =
torch.utils.data.DataLoader(dataset,batch_size=8,shuffle=True)
optimizer = torch.optim.Adam(model.parameters(),lr=1e-4)
criterion = nn.TripletMarginLoss(margin=1.0)
```

Evaluation Metrics: **Why Recall@K**?

Standard classification metrics (like Accuracy or F1 Score) are incompatible with Content-Based Image Retrieval (CBIR). In metric learning, the model is not predicting a discrete class; it is projecting an image into a continuous vector space.

To evaluate performance, I am using Recall@K (specifically Recall@5):

- **The Gallery Projection**: The entire validation dataset is passed through the network to generate a "gallery" of 256-dimensional embeddings.
- **The Query**: For each image, we calculate its Cosine Similarity (dot product of L_2 normalized vectors) against every other image in the gallery.
- **The Metric**: We retrieve the top $K=5$ nearest neighbors. If at least one of those 5 retrieved images shares the same ground-truth category as the query image, it is considered a "Hit".

Recall@5 is the percentage of total queries that resulted in a Hit. This perfectly simulates a real-world search engine where a user expects at least one highly relevant result on the first page of visual search results.

Training Loop

```
best_recall = 0.0
num_epoch = 30

for epoch in range(num_epoch):
    model.train()
```

```
    running_loss = 0
    running_pos = 0
    running_neg = 0

    for i,batch in enumerate(dataloader):
        anchor_img,pos_img,neg_img = batch
        anchor_img = anchor_img.to('cuda')
        pos_img = pos_img.to('cuda')
        neg_img = neg_img.to('cuda')

        a_emb = model(anchor_img)
        p_emb = model(pos_img)
        n_emb = model(neg_img)

        pos_dist = F.pairwise_distance(a_emb,p_emb)
        neg_dist = F.pairwise_distance(a_emb,n_emb)

        running_pos += pos_dist.mean().item()
        running_neg += neg_dist.mean().item()

        loss = criterion(a_emb,p_emb,n_emb)
        running_loss += loss.item()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {running_loss/len(dataloader):.4f}, \
          Pos Dist: {running_pos/len(dataloader):.4f}, Neg Dist: {running_neg/len(dataloader):.4f}")

    if((epoch+1)%10==0):
        gallery_emb, gallery_labels = [], []

        with torch.no_grad():
            model.eval()

            gd = GalleryDataset(final_df,transform=eval_transform)
            eval_loader = torch.utils.data.DataLoader(gd,batch_size=32,shuffle=False)

            for images,labels in eval_loader:
                images = images.to('cuda')

                embs = model(images)

                gallery_emb.append(embs.cpu())
                gallery_labels.append(labels)

            full_gallery = torch.cat(gallery_emb,dim=0)
```

```
            full_labels = torch.cat(gallery_labels,dim=0)

            # similarity matrix
            sim_matrix = torch.mm(full_gallery,full_gallery.t())
            sim_matrix.fill_diagonal_(-1.0)

            scores, indices = torch.topk(sim_matrix,k=5,dim=1)
            retrieved_labels = full_labels[indices]

            matches = retrieved_labels == full_labels.unsqueeze(1)
            hits = matches.any(dim=1)

            recall_at_5 = hits.float().mean().item()*100

            print(f"Recall@5: {recall_at_5:.2f}%")

            if recall_at_5 > best_recall:
                best_recall = recall_at_5
                torch.save(model.state_dict(),
'best_resnet_model.pth')
                print(f"--- Model Saved! New Best Recall:
{best_recall:.2f}% ---")

torch.save(model.state_dict(), 'final_resnet_model.pth')
print("Training complete. Final weights saved.")
```

Final Deterministic Evaluation (Recall@5)

```
model = EmbeddingNet().to('cuda')

model.load_state_dict(torch.load('final_resnet_model.pth',
weights_only=True))
print("Successfully loaded final_resnet_model.pth!")
model.eval()

gallery_emb, gallery_labels = [], []

print("Extracting gallery embeddings to calculate final Recall@5...")
with torch.no_grad():
    gd = GalleryDataset(final_df, transform=eval_transform)
    eval_loader = torch.utils.data.DataLoader(gd, batch_size=32,
shuffle=False)

    for images, labels in eval_loader:
        images = images.to('cuda')
        embs = model(images)

        gallery_emb.append(embs.cpu())
        gallery_labels.append(labels)
```

```
    full_gallery = torch.cat(gallery_emb, dim=0)
    full_labels = torch.cat(gallery_labels, dim=0)

    # Compute the Similarity Matrix
    sim_matrix = torch.mm(full_gallery, full_gallery.t())

    # Fill diagonal with -1 so an image doesn't match with itself
    sim_matrix.fill_diagonal_(-1.0)

    # Calculate Hits for Recall@5
    scores, indices = torch.topk(sim_matrix, k=5, dim=1)
    retrieved_labels = full_labels[indices]

    matches = retrieved_labels == full_labels.unsqueeze(1)
    hits = matches.any(dim=1)

    recall_at_5 = hits.float().mean().item() * 100

    print(f"\nFINAL DETERMINISTIC METRIC:")
    print(f"Recall@5: {recall_at_5:.2f}%")
```

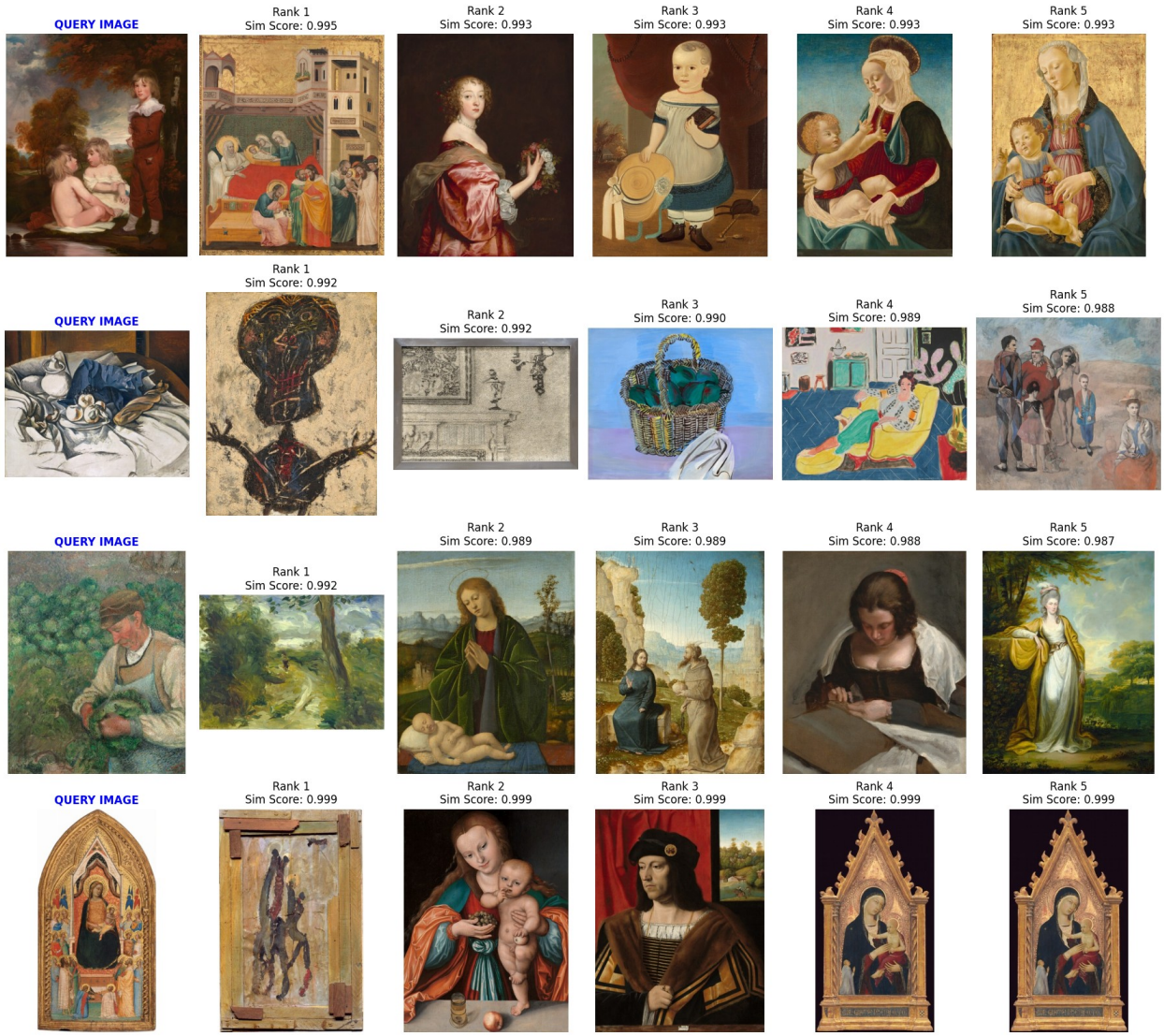
```
Successfully loaded final_resnet_model.pth!
Extracting gallery embeddings to calculate final Recall@5...

FINAL DETERMINISTIC METRIC:
Recall@5: 91.38%
```

Visual Inference

```
import matplotlib.pyplot as plt

def visualize_similarities(eval_model, df, transform, num_queries=3,
top_k=5):
    eval_model.eval()

    print("Extracting gallery embeddings for visualization...")
    gallery_dataset = GalleryDataset(df, transform=transform)
    loader = torch.utils.data.DataLoader(gallery_dataset,
batch_size=32, shuffle=False)

    all_embs = []
    with torch.no_grad():
        for imgs, _ in loader:
            imgs = imgs.to('cuda')
            all_embs.append(eval_model(imgs).cpu())

    gallery_embs = torch.cat(all_embs, dim=0) # Shape: [N, 256]

    fig, axes = plt.subplots(num_queries, top_k + 1, figsize=(18, 4 *
num_queries))
```

```
    for i in range(num_queries):
        # Pick a random query image
        q_idx = random.randint(0, len(df)-1)
        q_emb = gallery_embs[q_idx].unsqueeze(0)

        # Calculate Cosine Similarity (dot product works because
vectors are L2 Normalized)
        sims = torch.mm(q_emb, gallery_embs.t()).squeeze(0)

        # Mask out the query image itself so it doesn't return as the
100% match
        sims[q_idx] = -1.0

        scores, top_indices = torch.topk(sims, k=top_k)

        # Plot the Query Image
        q_img = Image.open(df.iloc[q_idx]['img_path'])
        axes[i, 0].imshow(q_img)
        axes[i, 0].set_title("QUERY IMAGE", fontweight='bold',
color='blue')
        axes[i, 0].axis('off')

        # Plot the Top-K Retrieved Images
        for j, rank_idx in enumerate(top_indices):
            res_img = Image.open(df.iloc[rank_idx.item()]['img_path'])
            axes[i, j+1].imshow(res_img)
            axes[i, j+1].set_title(f"Rank {j+1}\nSim Score:
{scores[j]:.3f}")
            axes[i, j+1].axis('off')

    plt.tight_layout()
    plt.show()

visualize_similarities(model, final_df, eval_transform, num_queries=4,
top_k=5)
```

```
Extracting gallery embeddings for visualization...
```



Model 2

Why **Vision Transformer (ViT-B/16)** as a Siamese Feature Extractor (Transfer Learning) ?

- **Global Self-Attention**: While CNNs build understanding locally pixel-by-pixel, the Vision Transformer evaluates the image holistically from the very first layer. This is highly effective for fine art, where "style" relies heavily on global composition and overarching color palettes rather than isolated edges.

- **The [CLS] Token Projection**: Instead of spatially pooling feature maps, the ViT slices the image into 16×16 patches. Through self-attention, a dedicated [CLS] token aggregates the stylistic context of the entire painting. By stripping the standard

ImageNet head and passing this token through a custom linear layer ($768 \rightarrow 256$), we project this global attention context directly onto our L_2 normalized hypersphere.

- **Transfer Learning (Warm Start)**: Initialized with ImageNet weights, the attention heads already possess a deep mathematical understanding of complex geometries, textures, and colors. Consequently, the Triplet Loss does not need to teach the network how to see; it only needs to optimize how to cluster what it already sees.
- **Isolated A/B Testing**: The geometric objective remains strictly identical to the ResNet baseline. By locking the exact same Triplet Margin Loss and L_2 normalization, we ensure a mathematically perfect, isolated A/B test comparing local Convolutional filters against global Self-Attention.

$$L = max(d(a,p) - d(a,n) + \alpha, 0)$$

```
class ViTEmbeddingNet(torch.nn.Module):
    def __init__(self):
        super().__init__()

        self.model = torchvision.models.vit_b_16(weights='DEFAULT')
        self.model.heads.head = nn.Linear(768,256)

    def forward(self,input):
        output = self.model(input)
        output = F.normalize(output,p=2,dim=1)
        return output
```

Model init

```
model = ViTEmbeddingNet().to('cuda')
dataloader =
torch.utils.data.DataLoader(dataset,batch_size=8,shuffle=True)
optimizer = torch.optim.Adam(model.parameters(),lr=1e-5)
criterion = nn.TripletMarginLoss(margin=1.0)
```

Training Loop

```
best_recall = 0.0
num_epoch = 30

for epoch in range(num_epoch):
    model.train()
    running_loss = 0
    running_pos = 0
    running_neg = 0

    for i,batch in enumerate(dataloader):
        anchor_img,pos_img,neg_img = batch
        anchor_img = anchor_img.to('cuda')
```

```
        pos_img = pos_img.to('cuda')
        neg_img = neg_img.to('cuda')

        a_emb = model(anchor_img)
        p_emb = model(pos_img)
        n_emb = model(neg_img)

        pos_dist = F.pairwise_distance(a_emb,p_emb)
        neg_dist = F.pairwise_distance(a_emb,n_emb)

        running_pos += pos_dist.mean().item()
        running_neg += neg_dist.mean().item()

        loss = criterion(a_emb,p_emb,n_emb)
        running_loss += loss.item()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {running_loss/len(dataloader):.4f}, \
          Pos Dist: {running_pos/len(dataloader):.4f}, Neg Dist: {running_neg/len(dataloader):.4f}")

    if((epoch+1)%10==0):
        gallery_emb, gallery_labels = [], []

        with torch.no_grad():
            model.eval()

            gd = GalleryDataset(final_df,transform=eval_transform)
            eval_loader = torch.utils.data.DataLoader(gd,batch_size=32,shuffle=False)

            for images,labels in eval_loader:
                images = images.to('cuda')

                embs = model(images)

                gallery_emb.append(embs.cpu())
                gallery_labels.append(labels)

            full_gallery = torch.cat(gallery_emb,dim=0)
            full_labels = torch.cat(gallery_labels,dim=0)

            # similarity matrix
            sim_matrix = torch.mm(full_gallery,full_gallery.t())
            sim_matrix.fill_diagonal_(-1.0)

            scores, indices = torch.topk(sim_matrix,k=5,dim=1)
```

```
            retrieved_labels = full_labels[indices]

            matches = retrieved_labels == full_labels.unsqueeze(1)
            hits = matches.any(dim=1)

            recall_at_5 = hits.float().mean().item()*100

            print(f"Recall@5: {recall_at_5:.2f}%")

            if recall_at_5 > best_recall:
                best_recall = recall_at_5
                torch.save(model.state_dict(),
'best_vit_metric_model.pth')
                print(f"--- Model Saved! New Best Recall:
{best_recall:.2f}% ---")

torch.save(model.state_dict(), 'final_vit_model.pth')
print("Training complete. Final weights saved.")
```

Final Deterministic Evaluation (Recall@5)

```
model = ViTEmbeddingNet().to('cuda')

model.load_state_dict(torch.load('final_vit_model.pth',
weights_only=True))
print("Successfully loaded final_vit_model.pth!")
model.eval()

gallery_emb, gallery_labels = [], []

print("Extracting gallery embeddings to calculate final Recall@5...")
with torch.no_grad():
    gd = GalleryDataset(final_df, transform=eval_transform)
    eval_loader = torch.utils.data.DataLoader(gd, batch_size=32,
shuffle=False)

    for images, labels in eval_loader:
        images = images.to('cuda')
        embs = model(images)

        gallery_emb.append(embs.cpu())
        gallery_labels.append(labels)

    full_gallery = torch.cat(gallery_emb, dim=0)
    full_labels = torch.cat(gallery_labels, dim=0)

    # Compute the Similarity Matrix
    sim_matrix = torch.mm(full_gallery, full_gallery.t())

    # Fill diagonal with -1 so an image doesn't match with itself
```

```
    sim_matrix.fill_diagonal_(-1.0)

    # Calculate Hits for Recall@5
    scores, indices = torch.topk(sim_matrix, k=5, dim=1)
    retrieved_labels = full_labels[indices]

    matches = retrieved_labels == full_labels.unsqueeze(1)
    hits = matches.any(dim=1)

    recall_at_5 = hits.float().mean().item() * 100

    print(f"\nFINAL DETERMINISTIC METRIC:")
    print(f"Recall@5: {recall_at_5:.2f}%")
```

```
Successfully loaded final_vit_model.pth!
Extracting gallery embeddings to calculate final Recall@5...

FINAL DETERMINISTIC METRIC:
Recall@5: 100.00%
```

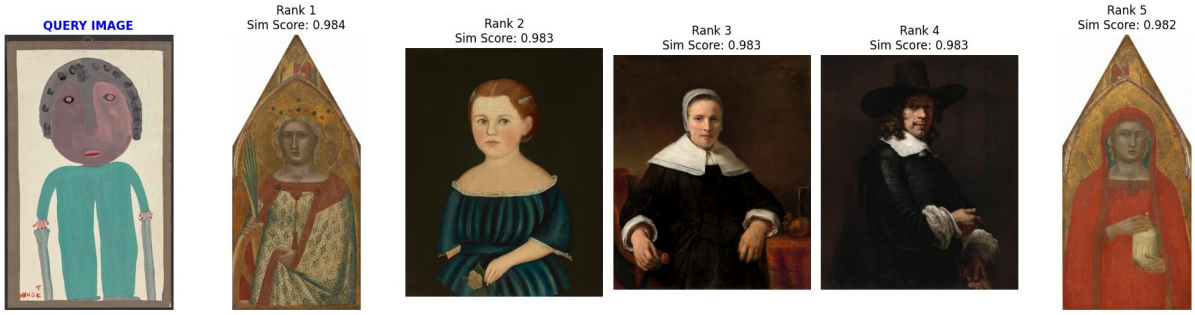
Visual Inference

```
visualize_similarities(model, final_df, eval_transform, num_queries=4,
top_k=5)
```

```
Extracting gallery embeddings for visualization...
```

QUERY IMAGE
Rank 1
Sim Score: 0.997
Rank 2
Sim Score: 0.997
Rank 3
Sim Score: 0.994
Rank 4
Sim Score: 0.994
Rank 5
Sim Score: 0.994

QUERY IMAGE
Rank 1
Sim Score: 0.951
Rank 2
Sim Score: 0.948
Rank 3
Sim Score: 0.947
Rank 4
Sim Score: 0.946
Rank 5
Sim Score: 0.942


QUERY IMAGE
Rank 1
Sim Score: 0.984
Rank 2
Sim Score: 0.983
Rank 3
Sim Score: 0.983
Rank 4
Sim Score: 0.983
Rank 5
Sim Score: 0.982

QUERY IMAGE
Rank 1
Sim Score: 0.988
Rank 2
Sim Score: 0.987
Rank 3
Sim Score: 0.986
Rank 4
Sim Score: 0.986
Rank 5
Sim Score: 0.986